

TP n°1 – API REST et Micro-services

Introduction

L'objectif de ce TP est de comprendre les principes fondamentaux des **API REST** et des **micro-services**, puis de les mettre en œuvre à travers des exemples simples.

Deux langages sont utilisés, **Java** et **Python**, afin de montrer que les concepts restent identiques indépendamment de la technologie choisie.

Dans une première partie, une API REST est implémentée en **Java avec Spring Boot**.

Dans une seconde partie, une API REST sera développée en **Python** avec persistance des données dans une **base de données relationnelle**.

Partie 1 : Notions théoriques

1. API REST

Une **API REST (Representational State Transfer)** est une interface permettant à des applications de communiquer via le protocole HTTP.

Elle repose sur des principes simples :

- utilisation des méthodes HTTP ([GET](#), [POST](#), [PUT](#), [DELETE](#))
- échange de données généralement au format **JSON**
- accès aux ressources via des **URI**

2. Micro-services

L'architecture **micro-services** consiste à découper une application en plusieurs services indépendants :

- chaque service a une responsabilité précise
- les services communiquent via des API REST
- ils peuvent être développés, déployés et maintenus séparément

Cette approche améliore la **scalabilité**, la **maintenabilité** et la **flexibilité** des applications.

3. Spring Boot

Spring Boot est un framework Java permettant de créer rapidement des applications web et des API REST :

- configuration minimale
 - serveur embarqué
 - annotations simples pour exposer des endpoints REST
-

Partie 2 : Manipulation – API REST en Java (Spring Boot)

1. Création du contrôleur REST

```
@RestController  
public class MyApi {
```

Interprétation :

- L'annotation `@RestController` indique que cette classe expose des **services REST**
 - Les méthodes de la classe retournent directement des données (texte ou JSON)
-

2. Endpoint `/b` – Message Bonjour

```
@GetMapping(value = "/b")  
public String bonjour(){  
    return "Bonjour !";  
}
```

Interprétation :

- `@GetMapping` associe cette méthode à une requête HTTP **GET**
- L'URL `/b` permet d'accéder à la ressource

- La méthode retourne une chaîne de caractères affichée directement dans le navigateur

3. Endpoint `/bn` – Message Bonsoir

```
@GetMapping(value = "/bn")
public String bonsoir(){
    return "Bonsoir";
}
```

Explication:

- Même principe que l'endpoint précédent
 - Permet de tester plusieurs routes REST dans la même API
 - Montre la simplicité de création d'API REST avec Spring Boot
-

4. Endpoint `/etudiant` – Retour d'un objet

```
@GetMapping(value = "/etudiant")
public Etudiant getEtudiant(){
    return new Etudiant(identifiant: 1, nom: "A", moyenne: 19);
}
```

Interprétation :

- Cet endpoint retourne un **objet Java**
- Spring Boot le convertit automatiquement en **JSON**
- Cela illustre le principe de sérialisation automatique